

**Ahmedabad
University**

AI-Powered Chatbot for University Policies and Documents

Project Report

M.Tech. CSE

Team Members:

Nishtha Mistry (AU2544001)

Under the Guidance of:

Dr. Maitrik Shah

School of Engineering and Applied Science
Ahmedabad University

Contents

Abstract	1
1 Introduction	2
1.1 Motivation	2
1.2 Problem Statement	2
2 Objectives	3
3 Background and Related Work	4
3.1 Traditional Keyword Search	4
3.2 Semantic Search and RAG	4
3.3 Sentence Transformers	4
3.4 FAISS	4
3.5 FLAN-T5	4
4 Dataset Description	5
4.1 Source of Documents	5
4.2 Dataset Summary	5
4.3 Categories	5
5 Methodology	6
5.1 System Architecture	6
5.2 Pipeline Steps	6
6 Implementation Details	8
6.1 Technology Stack	8
6.2 Hardware Environment	8
6.3 Directory Structure	8
6.4 UI Enhancements	8
7 Experiments and Results	9
7.1 Sample Queries and Outputs	9
7.2 Performance Observations	9
7.3 System Screenshot	9
8 Challenges and Limitations	10
9 Discussion	11
10 Conclusion	12

11 Future Work	13
Acknowledgements	14
References	15
Appendix	16

Abstract

University policy documents are long, complex, and difficult for students to search manually. Students often struggle to find straightforward information related to attendance rules, exam policies, grading systems, and registration requirements. To address this, we developed an AI-powered chatbot using a Retrieval-Augmented Generation (RAG) pipeline that extracts information from university policy PDFs and produces accurate, context-aware responses. The system includes document preprocessing, semantic chunking, embedding-based retrieval with FAISS, and answer generation using FLAN-T5. The chatbot is deployed through an interactive Streamlit interface. Experimental results demonstrate that the system significantly improves accessibility and reduces search time for students, providing fast and reliable answers to policy-related queries. **Such a system can be deployed as an official student assistant to streamline access to academic information across the university.**

1. Introduction

Universities maintain extensive policy documents that govern academic procedures, examination rules, attendance requirements, and administrative regulations. While these documents are essential, they are often lengthy, contain complex formatting, and are not user-friendly. Students frequently face difficulties in finding specific information within these large PDF files, leading to confusion and delays. For example, a student may spend several minutes scrolling through a 40-page document simply to find the rule on minimum attendance or the number of allowed absences.

This project aims to solve this challenge by developing an AI-powered chatbot capable of retrieving answers directly from university policy documents. Using modern techniques including embeddings, semantic search, and RAG architectures, the system transforms unstructured textual documents into searchable chunks and retrieves high-quality responses.

A total of **18 policy documents** were collected from the Auris student portal and used for testing the system.

1.1 Motivation

The primary motivation behind this project is to reduce the time and effort students spend searching for policy information. Traditional keyword search fails for queries that do not match exact keywords or require contextual understanding. Modern students expect fast, intuitive systems similar to ChatGPT. Hence, creating an AI policy assistant becomes both practical and aligned with modern student expectations.

1.2 Problem Statement

Students lack a quick and reliable way to query university policies. Existing PDFs are not optimized for search, keyword search is insufficient, and manual browsing is time-consuming.

How can we build a tool that enables students to instantly retrieve accurate, context-aware answers from long, multi-format university policy documents?

2. Objectives

- To collect and preprocess university policy documents from multiple formats.
- To design a chunking strategy that preserves semantic meaning.
- To generate contextual embeddings using Sentence Transformers.
- To build a FAISS-based vector store for efficient retrieval.
- To retrieve relevant policy chunks based on a user's query.
- To generate final answers using FLAN-T5.
- To implement a clean, user-friendly Streamlit interface for student interaction.

3. Background and Related Work

3.1 Traditional Keyword Search

Keyword search relies on exact word matching. This approach is insufficient for academic policy documents because:

- It requires exact wording.
- It fails when synonyms or paraphrased queries are used.
- It cannot interpret meaning or intent.

3.2 Semantic Search and RAG

Retrieval-Augmented Generation (RAG) improves search by retrieving relevant chunks and using an LLM to generate the final answer.

3.3 Sentence Transformers

We use `all-mpnet-base-v2`, known for high semantic similarity performance and 768-dimensional embeddings.

3.4 FAISS

FAISS provides efficient large-scale similarity search with inner-product similarity.

3.5 FLAN-T5

FLAN-T5 Base (250M parameters) is used for final answer generation due to its lightweight, fast, and instruction-tuned nature.

4. Dataset Description

4.1 Source of Documents

All documents were collected from the official Auris portal of Ahmedabad University.

4.2 Dataset Summary

The dataset consists of:

- **18 policy documents**
- PDF files (policy manuals, academic calendars)
- DOCX files (guidelines, rules)

4.3 Categories

- Attendance rules
- Examination & grading regulations
- Course registration procedures
- Tuition fee policies
- Academic calendar policies

5. Methodology

5.1 System Architecture

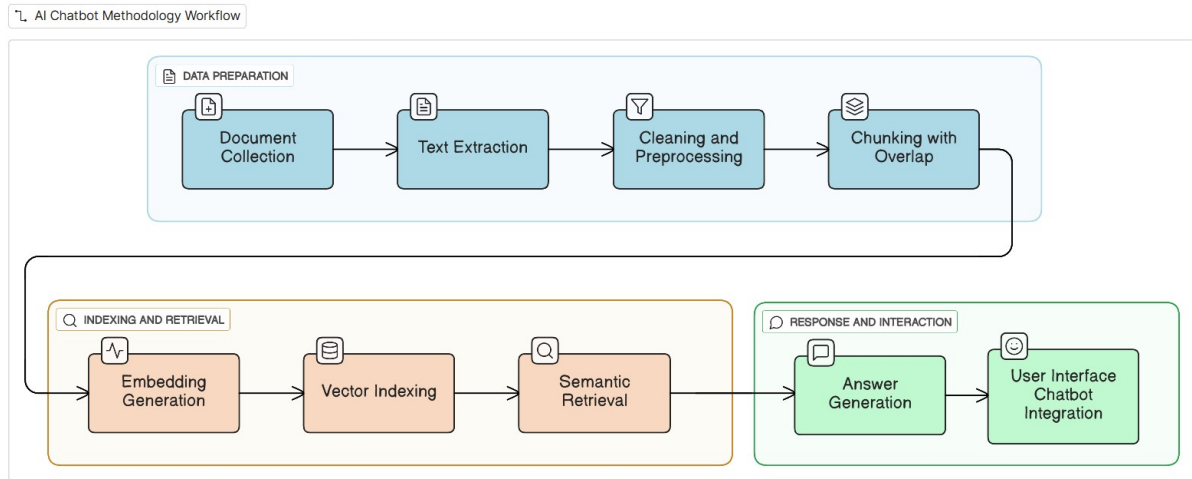


Figure 5.1: Overall Workflow of the System

5.2 Pipeline Steps

1. **Document Collection** All PDF and DOCX files were gathered from the Auris portal.
2. **Text Extraction** Using:
 - `pdfplumber` for PDFs
 - `python-docx` for DOCX files
3. **Cleaning and Preprocessing** Text preprocessing involved removing special characters, fixing line breaks, and normalizing whitespace.
4. **Chunking** We implemented 200-word chunks with 50-word overlap.
5. **Embedding Generation** Embeddings were generated using the `all-mpnet-base-v2` model.
6. **FAISS Vector Store** FAISS `IndexFlatIP` was used for efficient similarity search.
7. **Retrieval Mechanism** Top-k similarity search retrieves the most relevant chunks.
8. **Answer Generation** FLAN-T5 Base was used to summarize retrieved chunks into a final coherent answer.

9. **Streamlit Interface** A responsive and intuitive user interface was developed using Streamlit. The interface allows users to enter queries, view retrieved results, and interact with the system in real time. Streamlit's session state was used to manage variables and maintain consistent application behavior during reruns.

6. Implementation Details

6.1 Technology Stack

- Python 3.10
- pdfplumber
- python-docx
- Sentence Transformers
- FAISS
- Transformers (FLAN-T5)
- Streamlit

6.2 Hardware Environment

Tested on:

- Intel i5 CPU
- 16GB RAM
- No GPU required

6.3 Directory Structure

```
project/  
  data/  
  embeddings/  
  app.py  
  utils/
```

6.4 UI Enhancements

- Preserved FLAN-T5 checkbox value using session state
- Clean minimal chatbot mode for demonstrations

7. Experiments and Results

7.1 Sample Queries and Outputs

User Query	Chatbot Answer
“What happens if fees are not paid on time?”	The student is assessed for a negligent fee and a Student Accounts hold is placed on the record until charges are paid in full.
“What happens if student has an F/NP grade in any course?”	The course must be repeated when it is offered next and the ‘F / NP’ must be cleared.
“What happens in case of any academic dishonesty?”	Examination / Discipline Committee of the school holds the discretion of taking suitable action against the student.

7.2 Performance Observations

- Retrieval was fast
- FLAN-T5 produced coherent responses
- Slight difficulty with table-heavy documents

7.3 System Screenshot

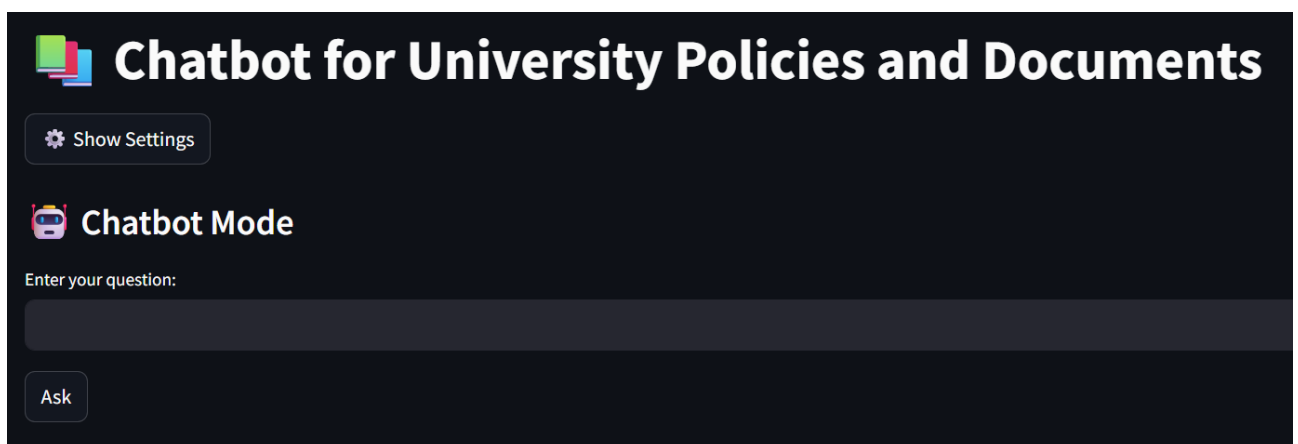


Figure 7.1: Streamlit Chatbot Interface

8. Challenges and Limitations

- Complex table structures caused extraction issues.
- Finding the optimal chunk size required experimentation.
- FLAN-T5 occasionally paraphrased answers.
- Session state had to be managed carefully due to Streamlit reruns.

9. Discussion

The system performed significantly better than keyword-based search. Semantic retrieval ensured even paraphrased queries were correctly interpreted, while RAG improved contextual accuracy. However, limitations in table extraction and minor LLM hallucinations indicate opportunities for future enhancement.

10. Conclusion

The project successfully demonstrates an AI-powered chatbot capable of retrieving and summarizing university policy information. By integrating document preprocessing, semantic chunking, embedding-based retrieval, and LLM-driven answer generation, the system provides a fast, intuitive way for students to access academic rules. FAISS enabled efficient retrieval, while FLAN-T5 produced high-quality answers even on CPU hardware.

11. Future Work

- Enhance table extraction using LLaMA/TAPAS.
- Use a larger LLM (FLAN-T5 XL, LLaMA 3).
- Deploy as an official university assistant.
- Add multilingual support.
- Include user feedback rating to improve answer quality.

Acknowledgements

We express our heartfelt gratitude to our faculty guide, Dr. Maitrik Shah, for his invaluable guidance, mentorship, and support throughout the duration of this project. We also extend our thanks to our classmates and peers for their constructive feedback and encouragement during development and testing.

References

- FAISS: Facebook AI Similarity Search. <https://faiss.ai>
- Sentence Transformers. <https://www.sbert.net>
- Streamlit Framework. <https://streamlit.io>
- HuggingFace Transformers. <https://huggingface.co>
- Google FLAN-T5 Model Card. <https://huggingface.co/google/flan-t5-base>

Appendix

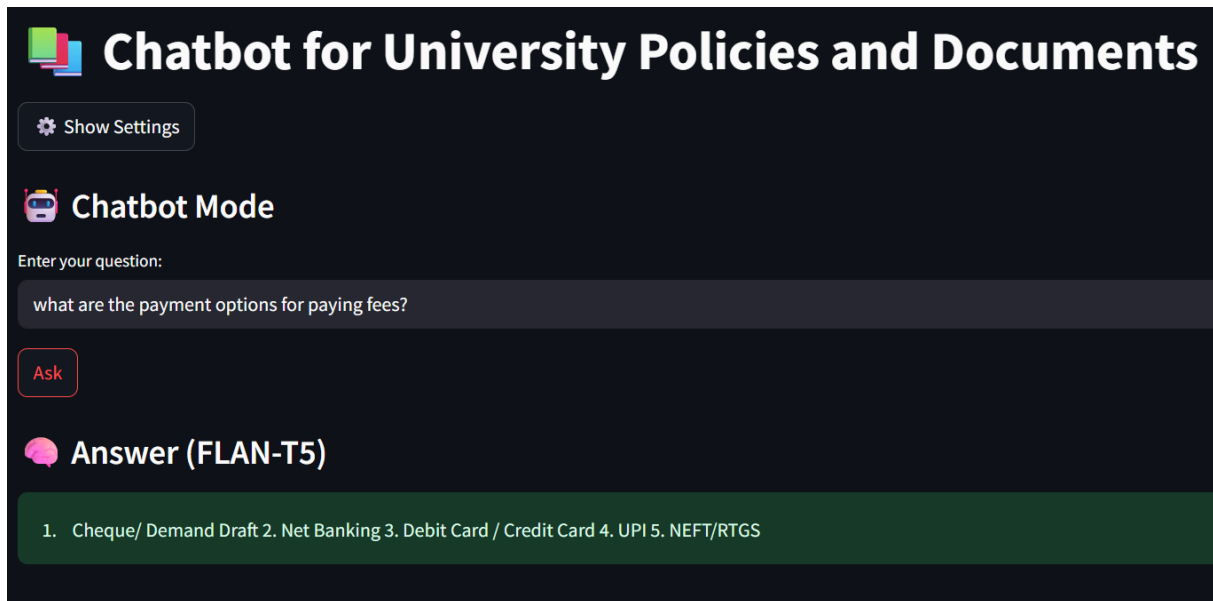


Figure 11.1: Output Sample - 1

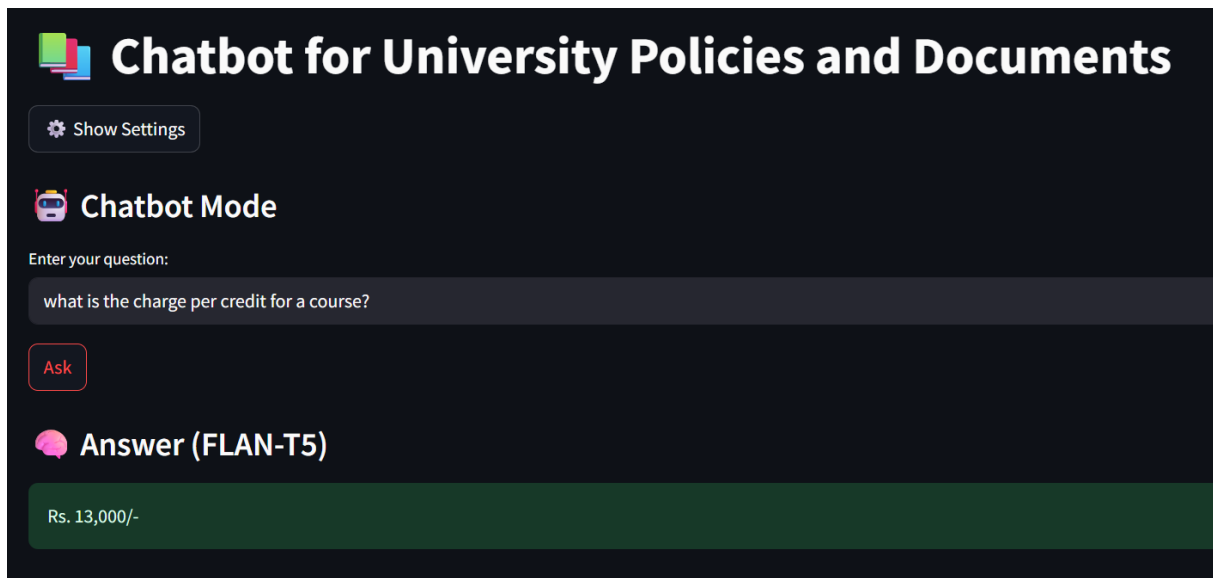


Figure 11.2: Output Sample - 2